EE789:Algorithmic Design of Digital Systems
Assignment 2 : Problem 1 Report

Name of student : Prathmesh Baral
Roll no : 23B1260

April 5, 2026

---

1 Problem Statement

Verify that the base implementation shared with you works correctly

2 Program Explanation

At the beginning of the program, modules are defined to initialize and access the matrices A, B, and the result matrix RESULT. The modules `init_A_entry` and `init_B_entry` are used to load values into matrices A and B respectively. Similarly, `get_A_entry`, `get_B_entry`, and `get_result_entry` are used to read values from these matrices.

A global counter is then defined using the signal GLOBAL_COUNTER. The module `counter_daemon` continuously increments this counter at every clock cycle. Since it is declared globally, it can be accessed from any module to measure execution time in terms of clock ticks.

The top-level module `mmul` performs matrix multiplication. At the start of execution, it records the initial time using the global counter. It then iterates over all indices (I, J) of the result matrix using nested loop constructs implemented through branch blocks and ϕ variables.

For each element RESULT$[I][J]$, the module calls the `dotp` function. This function computes the dot product of the I^{th} row of matrix A and the J^{th} column of matrix B. Once all elements are computed, the module exits the loop and calculates the total number of clock ticks as:

$$\texttt{nticks} = \text{GLOBAL\_COUNTER} - \texttt{start\_time}$$

The `dotp` module is responsible for computing the dot product. It iterates over index K, multiplying corresponding elements $A[I][K]$ and $B[K][J]$, and accumulates the result in a running sum. This process is pipelined for efficiency. After completing all iterations, the final accumulated sum is returned as the output.

Overall, the program implements matrix multiplication using a modular and pipelined approach, while also measuring execution time using a global counter.

3 Tesing with N = 8

3.1 Test 1: Multiplication of Identity Matrices ($A = B = I$)

The output corresponding to this test is stored in `Output_1.log`, and the associated testbench file is `testbench_1.c`. The screenshot of the terminal output is shown in Fig. 1.
Ticks needed for computation : 5913

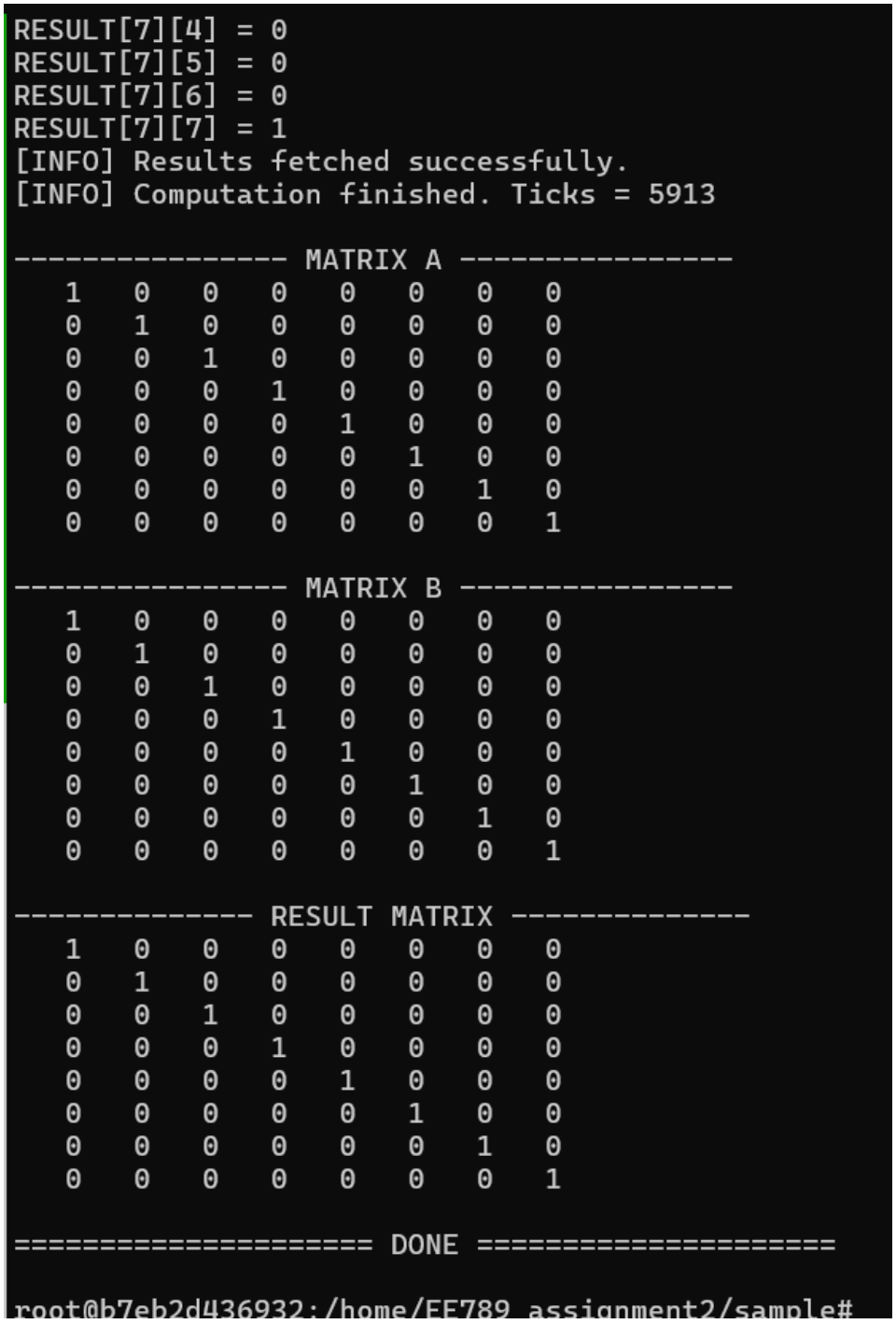

Figure 1: Terminal output for Test 1

3.2 Test 2: Matrix with $A[i][j] = i + j + 1$ and $B = I$

In this test, matrix A is defined such that each element follows the relation $A[i][j] = i + j + 1$, resulting in values ranging from 1 to 15, while B is the identity matrix.

The output corresponding to this test is stored in `Output_2.log`, and the associated testbench file is `testbench_2.c`. The screenshot of the terminal output is shown in Fig. 2.

Ticks needed : 5913

```
RESULT[7][4] = 12
RESULT[7][5] = 13
RESULT[7][6] = 14
RESULT[7][7] = 15
[INFO] Results fetched successfully.
[INFO] Computation finished. Ticks = 5913

---------------- MATRIX A ----------------
   1   2   3   4   5   6   7   8
   2   3   4   5   6   7   8   9
   3   4   5   6   7   8   9  10
   4   5   6   7   8   9  10  11
   5   6   7   8   9  10  11  12
   6   7   8   9  10  11  12  13
   7   8   9  10  11  12  13  14
   8   9  10  11  12  13  14  15

---------------- MATRIX B ----------------
   1   0   0   0   0   0   0   0
   0   1   0   0   0   0   0   0
   0   0   1   0   0   0   0   0
   0   0   0   1   0   0   0   0
   0   0   0   0   1   0   0   0
   0   0   0   0   0   1   0   0
   0   0   0   0   0   0   1   0
   0   0   0   0   0   0   0   1

-------------- RESULT MATRIX --------------
   1   2   3   4   5   6   7   8
   2   3   4   5   6   7   8   9
   3   4   5   6   7   8   9  10
   4   5   6   7   8   9  10  11
   5   6   7   8   9  10  11  12
   6   7   8   9  10  11  12  13
   7   8   9  10  11  12  13  14
   8   9  10  11  12  13  14  15

==================== DONE ====================
```

Figure 2: Terminal output for Test 2

3.3 Test 3: Matrices with $A[i][j] = i$ and $B[i][j] = j$

In this test, matrix A is defined such that each element depends only on the row index ($A[i][j] = i$), while matrix B depends only on the column index ($B[i][j] = j$).

The output corresponding to this test is stored in `Output_3.log`, and the associated testbench file is `testbench_3.c`. The screenshot of the terminal output is shown in Fig. 3.

Ticks needed : 5913

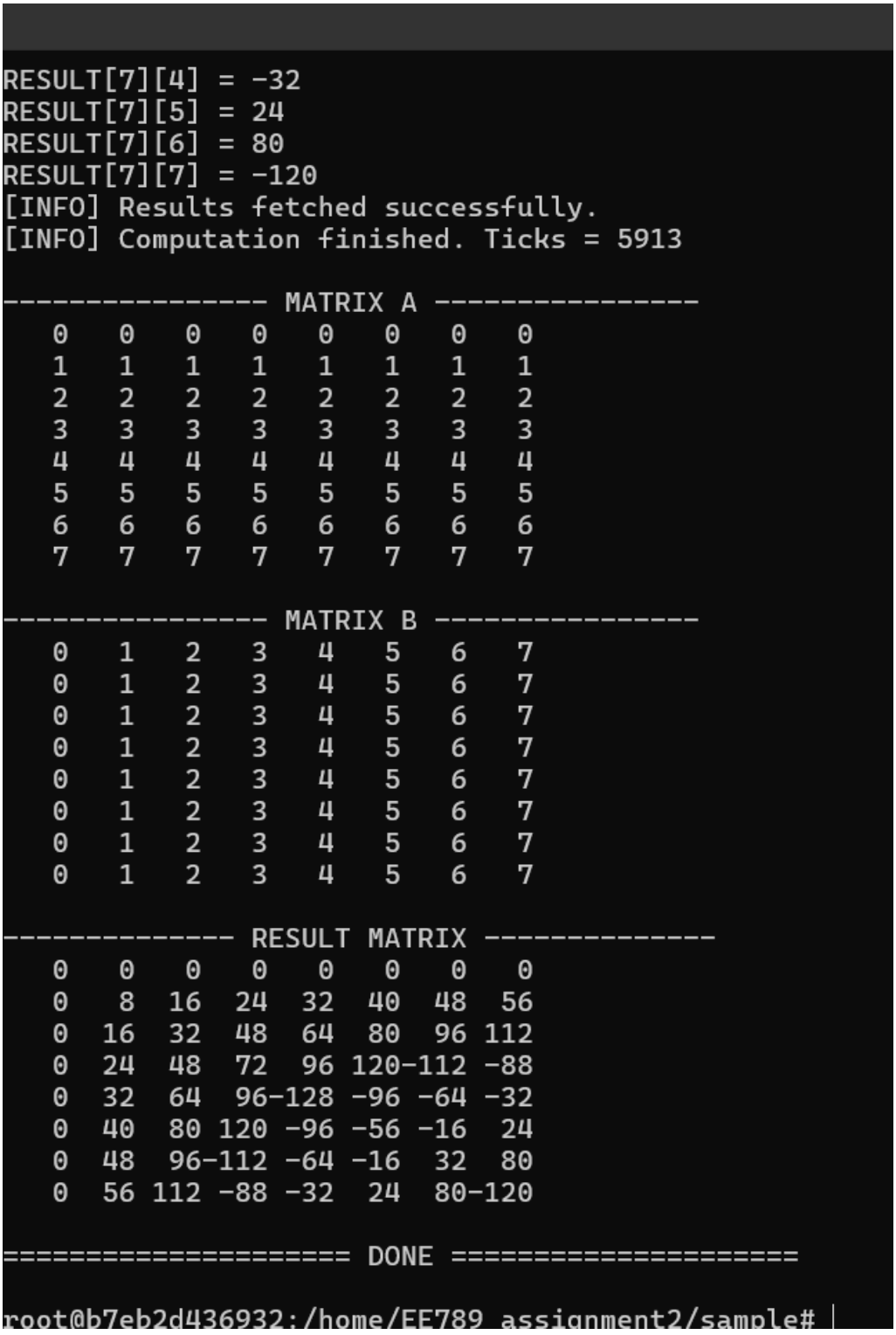

Figure 3: Terminal output for Test 3

Note: The computed values are clipped to 8-bit signed representation. Hence, overflow leads to wrapping, which results in negative values appearing in the output.

3.4 Test 4: Reduced Range Matrices with $A[i][j] = i \bmod 4$ and $B[i][j] = j \bmod 4$

To avoid overflow observed in the previous test, the input matrices are modified such that $A[i][j] = i \bmod 4$ and $B[i][j] = j \bmod 4$. This ensures that all computed values remain within the 8-bit signed range.

The output corresponding to this test is stored in `Output_4.log`, and the associated testbench file is `testbench_4.c`. The screenshot of the terminal output is shown in Fig. 4.

Ticks needed : 5913

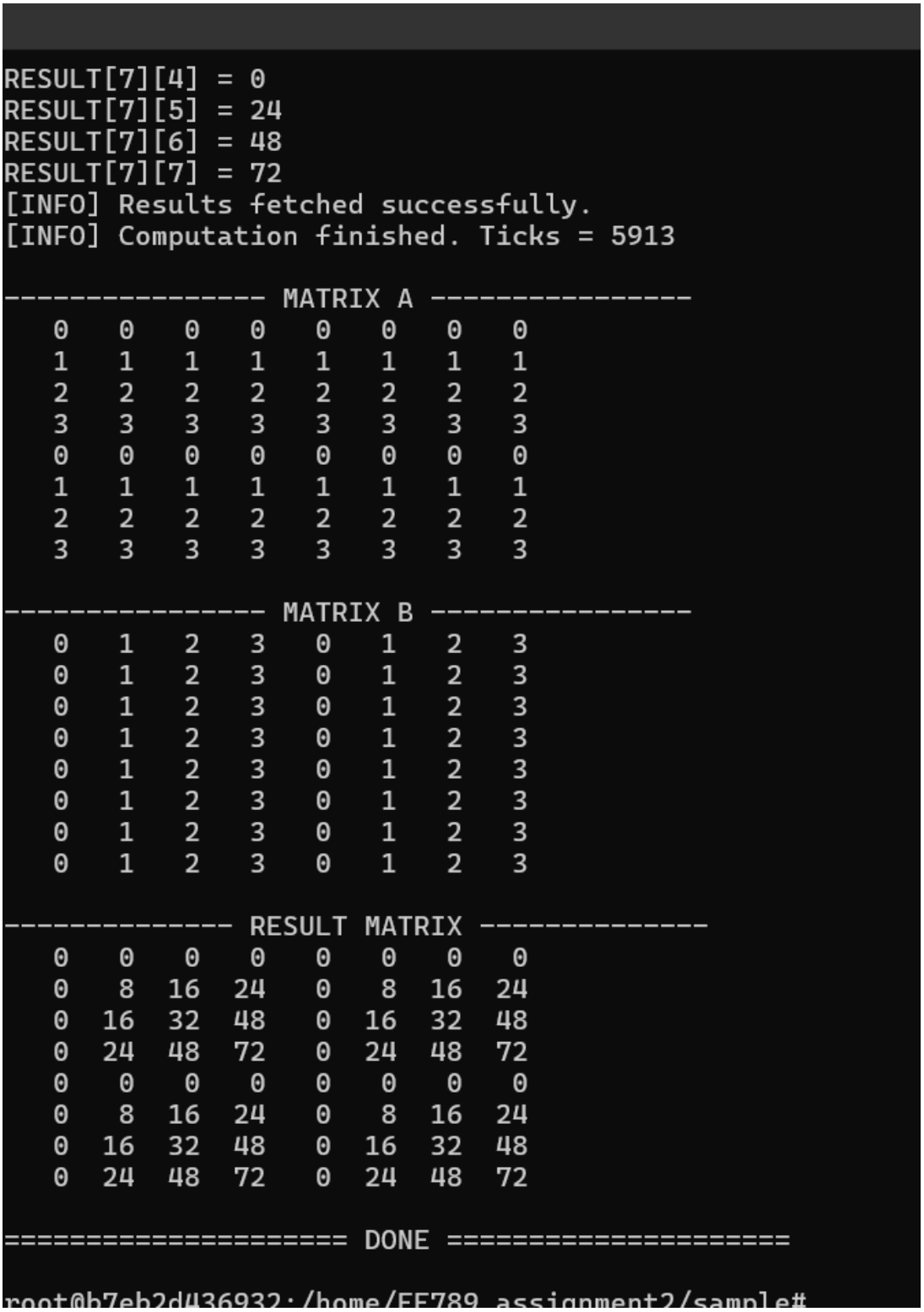

Figure 4: Terminal output for Test 4

4 Testing with N = 64

4.1 Test 1: Multiplication of Identity Matrices ($A = B = I$)

The output corresponding to this test is stored in `Output_1.log`, and the associated testbench file is `testbench_1.c`. The screenshot are attached below
Ticks needed for computation : 2670785

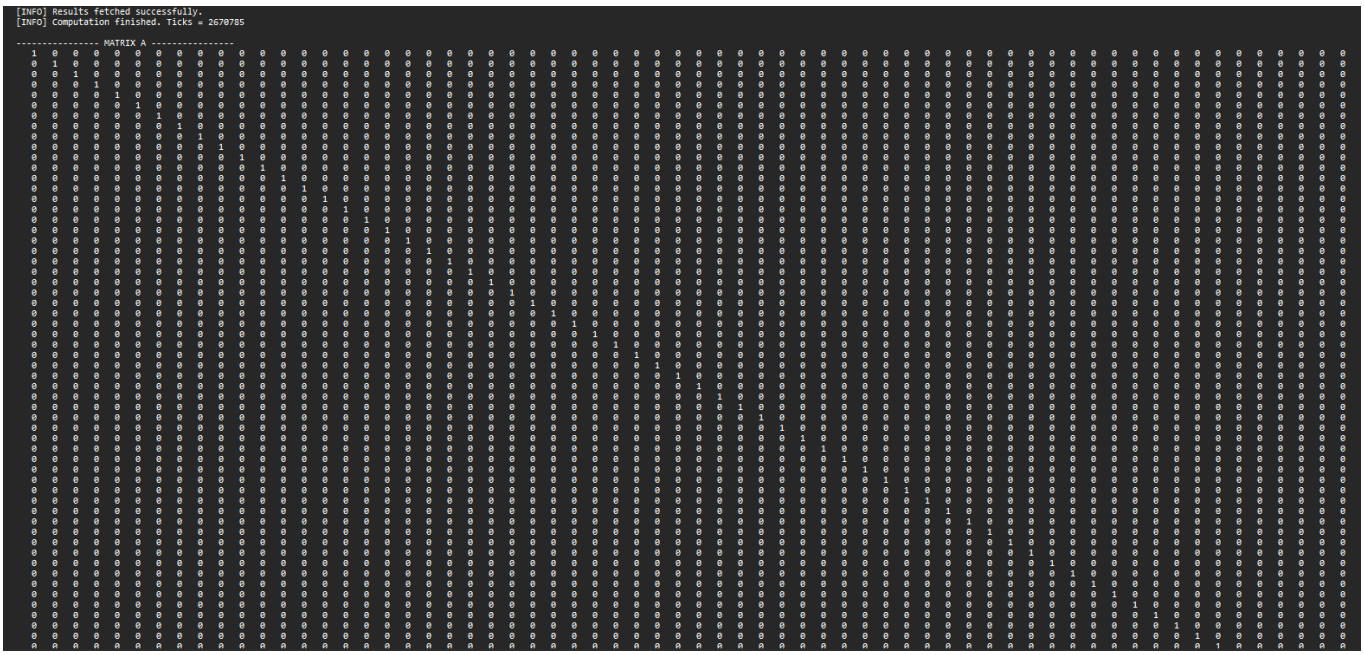

Figure 5: Terminal output for Test 1 : matrix A

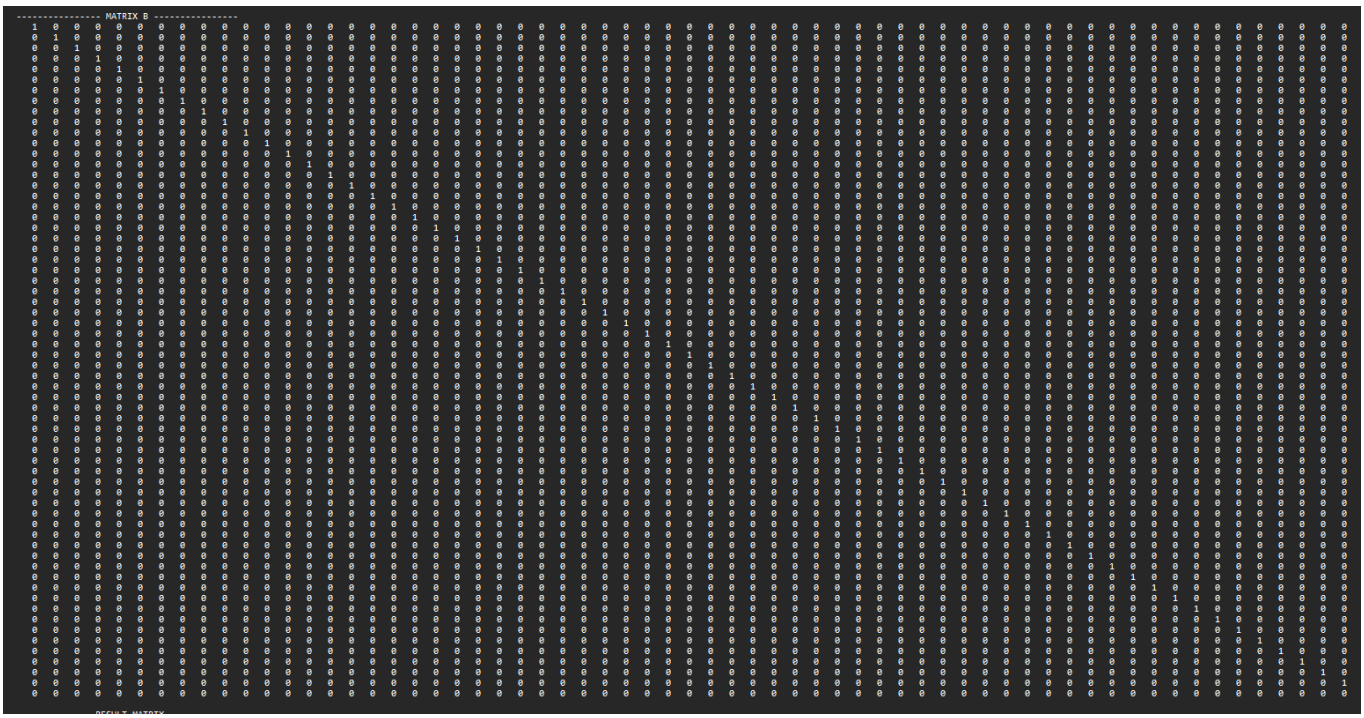

Figure 6: Terminal output for Test 1 : matrix B

Figure 7: Terminal output for Test 1 : Result

4.2 Test 2: Matrix with $A[i][j] = i + j$ and $B = I$

In this test, matrix A is defined such that each element follows the relation $A[i][j] = i + j$, resulting in values ranging from 0 to 126, while B is the identity matrix.

The output corresponding to this test is stored in `Output_2.log`, and the associated testbench file is `testbench_2.c`. The screenshot of the terminal output are below.

Ticks needed : 2670785

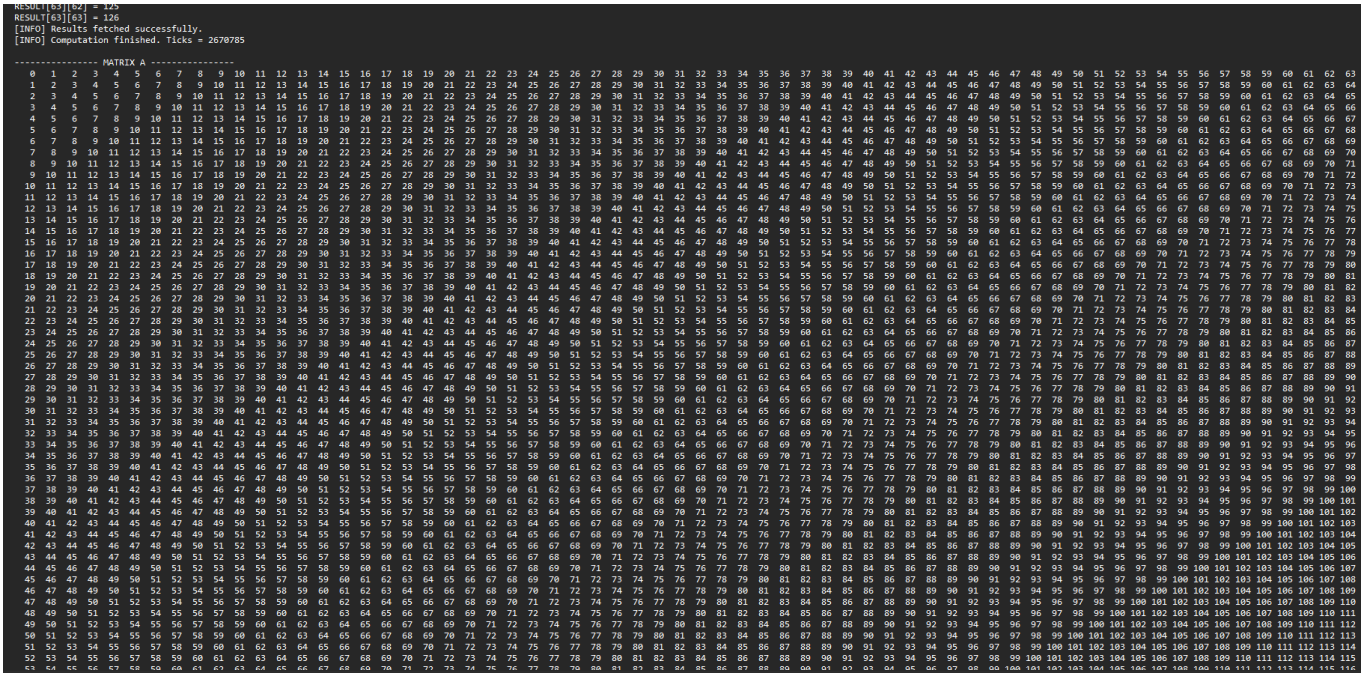

Figure 8: Terminal output for Test 1 : matrix A

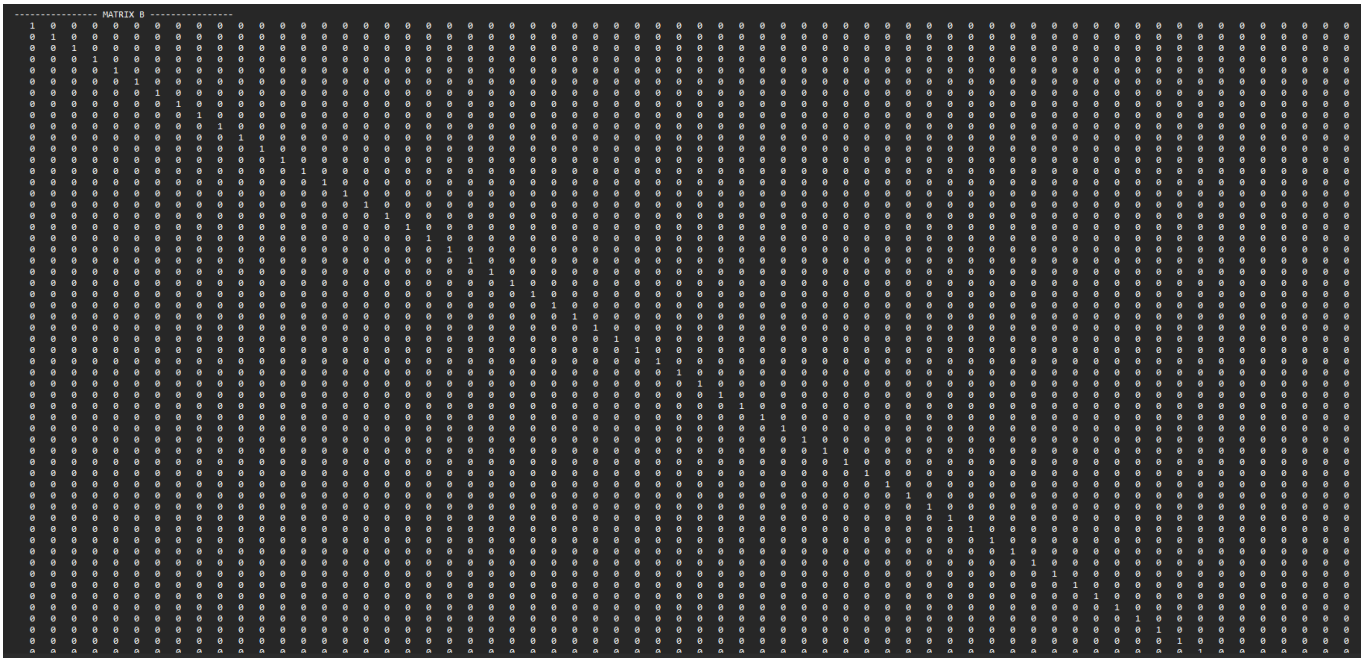

Figure 9: Terminal output for Test 1 : matrix B

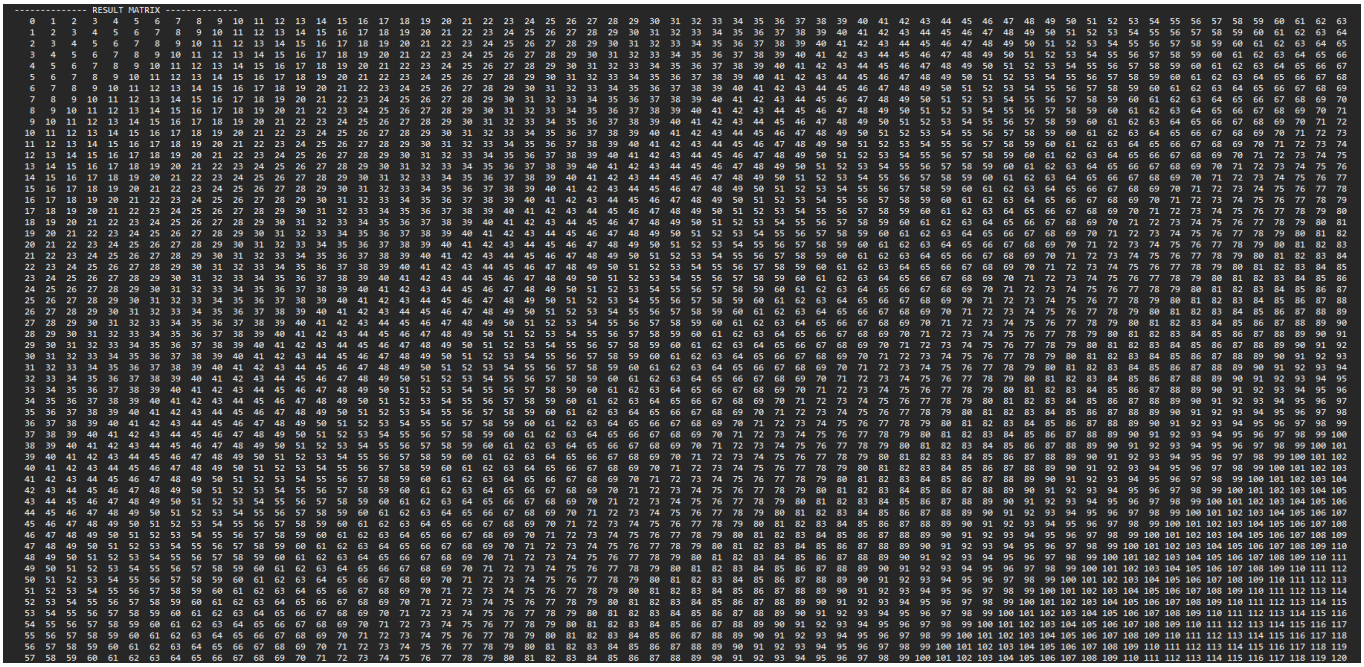

Figure 10: Terminal output for Test 1 : Result

5 Conclusion

In this work, four test cases were successfully executed for 8×8 matrix multiplication, and the outputs matched the expected results. Each computation required 5913 clock ticks, demonstrating correct functionality and consistent performance for the base implementation.

Further, a larger test case involving 64×64 matrix multiplication was attempted. However, the execution time was significantly high, exceeding one hour for one test. Due to this constraints, only two test cases was executed, which produced correct results. The observed tick count for this case was approximately 2,670,785, indicating a substantial increase in computational cost compared to the 8×8 case.

Overall, while the implementation is functionally correct, performance scalability for larger matrix sizes is limited and requires optimization.